Bash Scripting

Table of Contents

1. Basic Bash Syntax
2. Creating a Bash Script
3. Bash Variables
4. Bash Data Types
5. Bash Operators
6. If...Else Statements
7. Bash Loops
8. Bash Functions

1. Basic Bash Syntax

Bash ("Bourne Again SHell") reads a script line by line and runs each line as a command. Before writing any script, it helps to understand the small set of rules that shape every line: how comments work, what order commands run in, how to separate them, and the general shape of a command itself.

1.1 Anatomy of a Command

Every command typically follows the same shape: a command name, optional options (flags), and optional arguments.

```
command  -options  argument1  argument2

# Example:
ls  -l  /home/user
#  ^cmd  ^option   ^argument
```

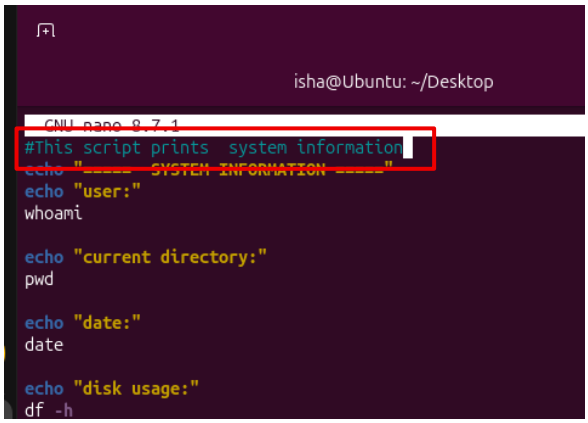
Every command that finishes returns an exit status (also called a return code): 0 means success, and any non-zero value (1–255) means some kind of failure. This value is stored automatically in the special variable $?.

```
ls /home/user
echo $?      # prints 0 if the ls command succeeded
```

1.2 Comments

A comment starts with a # character. The shell ignores everything after it on that line, so comments are used to explain what the code does without affecting execution. Comments can be on their own line or placed after a command (inline).

```
# This script prints a greeting message
echo "Hello, World!"   # inline comment explaining this line
```

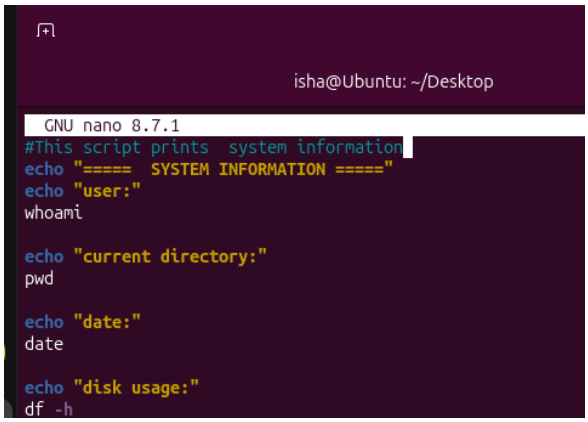


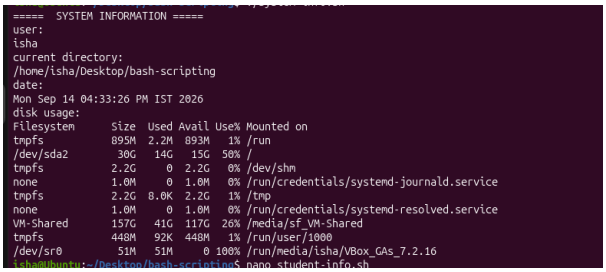
> **Note:**
> A # inside quotes is not treated as a comment — it is just a character. For example echo "Price: #5" prints the # normally.

1.3 Command Execution Order

Commands in a script run one after another, from top to bottom. This top-to-bottom order determines the script's logic and behaviour — Bash does not skip ahead or reorder commands on its own.

```
echo "First command"
echo "Second command"
```





1.4 Semicolons and Line Continuation

A semicolon (;) separates multiple commands placed on a single line, which is handy for writing short, concise one-liners. A single command that is too long to read comfortably can also be split across multiple lines using a backslash (\) at the end of the line.

```
# Semicolon: two commands on one line
echo "This is a test"; echo "This is another test"

# Backslash: one long command spread over two lines
echo "This message is quite long, so it \
   continues onto the next line"
```

1.5 Whitespace and Quoting Basics

Bash is sensitive to spaces in ways many other languages are not. Variable assignment must have no spaces around the = sign, and unquoted values are split on whitespace, which can change how a command behaves.

Style	Example	Result
Correct assignment	name="Alex"	Works — no spaces around =
Incorrect assignment	name = "Alex"	Error — Bash treats 'name' as a command
Unquoted variable	echo $sentence	Word-splits on spaces; multiple lines collapse

Style	Example	Result
Quoted variable	echo "$sentence"	Preserves spacing and line breaks exactly

1.6 Good Practice

- Use comments to explain non-obvious code, not what is already obvious from the command itself.
- Choose clear, meaningful variable names (total_files rather than tf).
- Always quote variables ("$var") unless you specifically want word-splitting.
- Test scripts thoroughly, including edge cases, before relying on them.

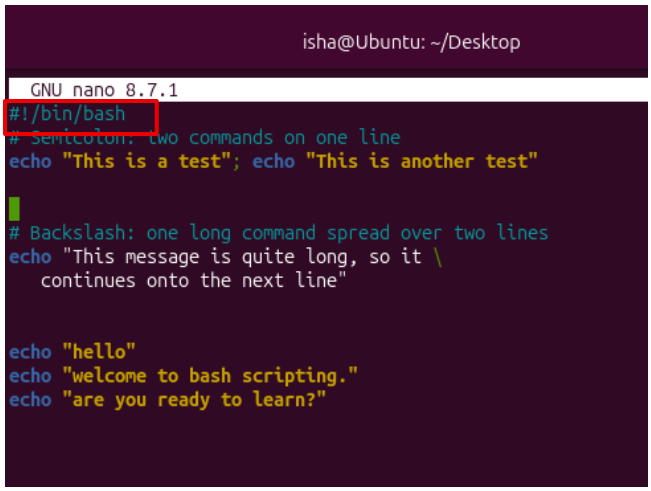
2. Creating a Bash Script

A Bash script is simply a plain text file containing a sequence of commands that the shell executes when the file is run. Turning a set of commands into a reusable script involves a few standard steps.

2.1 The Shebang Line

Every script should begin with a shebang (#!) followed by the absolute path to the interpreter that should run it — typically /bin/bash. The shebang must be the very first line of the file; if it is missing, the script will instead be run by whatever shell is currently active, which can cause subtle inconsistencies.

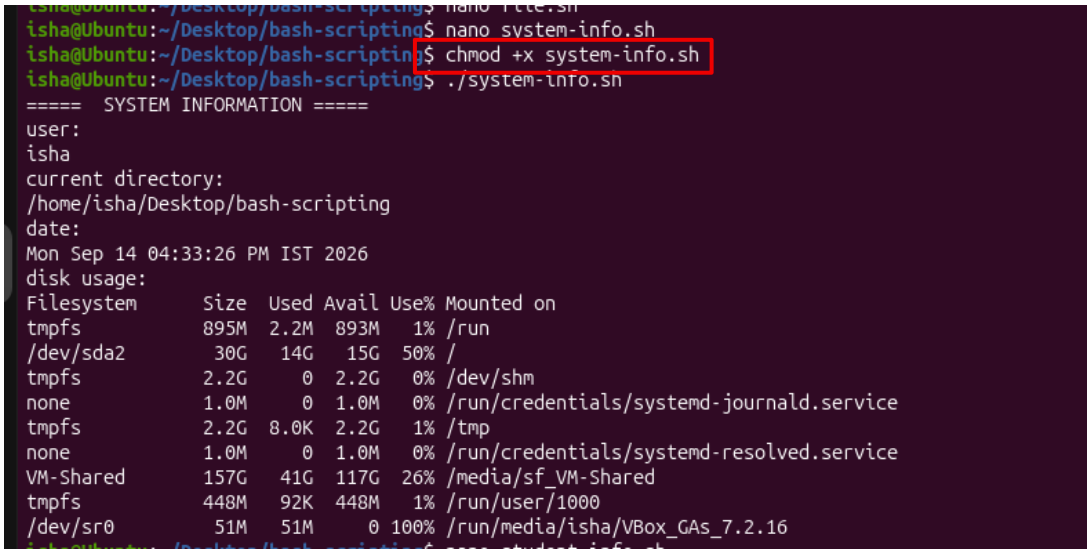
```
#!/bin/bash
# This script prints a greeting message
echo "Hello, World!"
```



2.2 Making a Script Executable

A newly created script file does not have permission to run as a program by default. The chmod command adds execute permission, after which the script can be launched directly.

```
chmod +x myscript.sh    # add execute permission
./myscript.sh           # run it (./ tells the shell to look in the current folder)
```



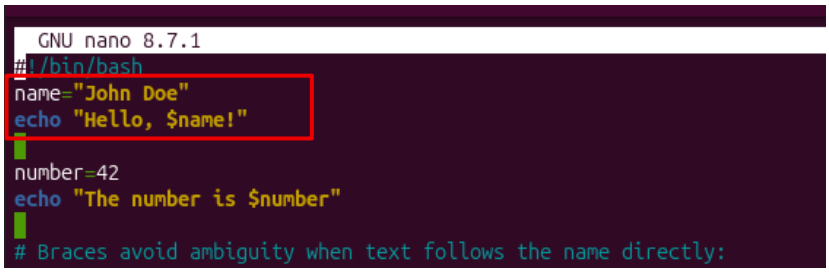
2.3 Ways to Run a Script

Command	What it does
./script.sh	Runs the script using the interpreter named in its shebang (needs execute permission).
bash script.sh	Explicitly runs the file with Bash, even without execute permission.
sh script.sh	Runs it with the system's default shell (may not support all Bash-only features).
source script.sh (or . script.sh)	Runs the script in the current shell session, so any variables it sets remain available afterward.

2.4 Using Variables in a Script

Variables let a script store and reuse data. A value is assigned with = (no spaces around it) and read back by prefixing the name with $.

```
#!/bin/bash
# Assign a value to a variable
name="World"
echo "Hello, $name!"
```



2.5 Passing Arguments to a Script

A script can accept input from the command line. Bash automatically stores these inputs in numbered positional variables.

Variable	Meaning
$0	The name of the script itself
$1, $2, $3 ...	The first, second, third, ... argument passed to the script
$#	The total number of arguments passed
$@	All arguments, treated as separate words
$*	All arguments, treated as a single combined string

```
#!/bin/bash
echo "Script name: $0"
echo "First argument: $1"
echo "Total arguments: $#"

# Run with:  ./myscript.sh apple banana
```

2.6 Script Structure — Good Practice

- Start with the shebang line.
- Add a short comment block describing what the script does and how to run it.
- Group related logic into functions (covered in Section 8) once a script grows beyond a few lines.
- Use exit 0 (success) or exit 1 (failure) at the end to signal the script's outcome to whatever called it.

3. Bash Variables

Variables store data that a script can manipulate. Bash variables are untyped — the same variable can hold text, a number, or the output of another command, and Bash will interpret it based on context.

3.1 Declaring and Accessing Variables

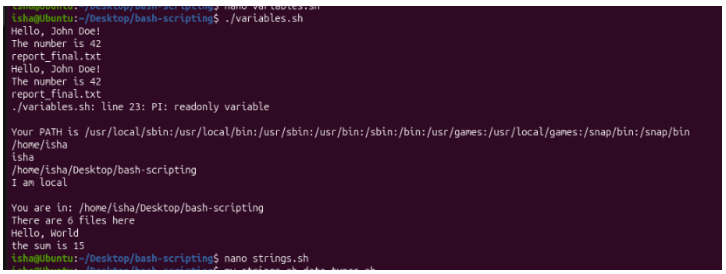
A variable is declared by assigning it a value directly (no spaces around =). Its value is read by prefixing the name with a dollar sign, or by wrapping the name in ${ } — the braces form is safer when the variable name is immediately followed by other text.

```
name="John Doe"
echo "Hello, $name!"

number=42
echo "The number is $number"

# Braces avoid ambiguity when text follows the name directly:
file="report"
echo "${file}_final.txt"   # -> report_final.txt
```



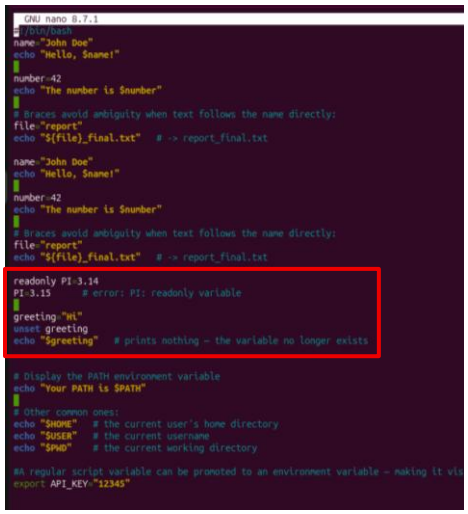


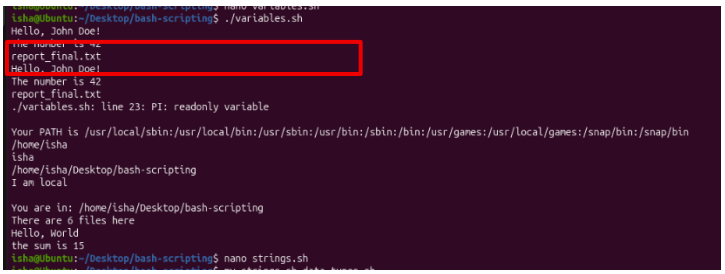
3.2 Read-only and Unsetting Variables

A variable can be locked so its value cannot be changed later using readonly, and removed entirely from the environment using unset.

```
readonly PI=3.14
PI=3.15      # error: PI: readonly variable

greeting="Hi"
unset greeting
echo "$greeting"   # prints nothing — the variable no longer exists
```



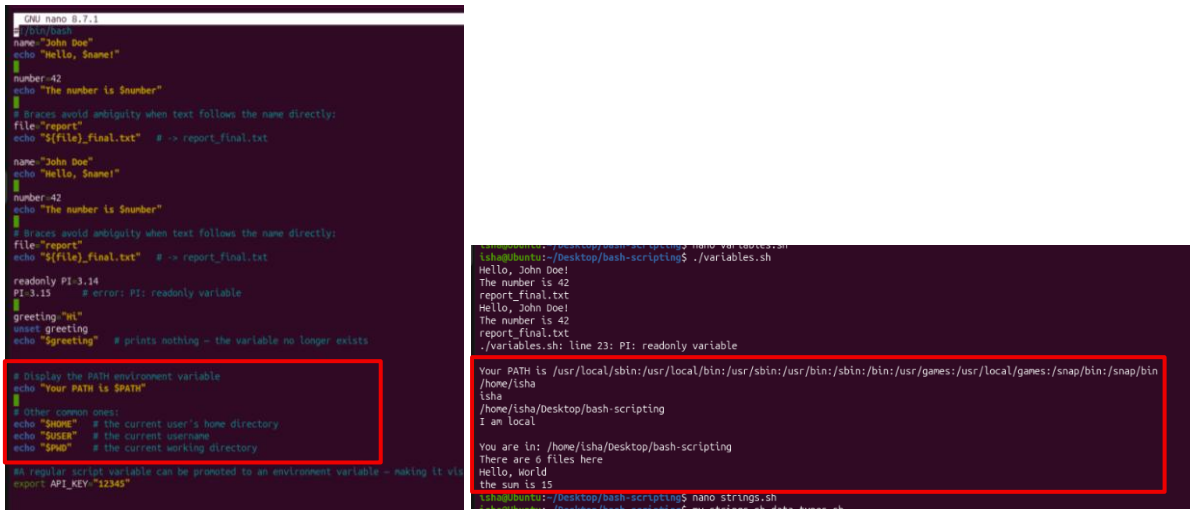


3.3 Environment Variables

Environment variables are predefined, system-wide variables that influence how processes behave. They are conventionally written in UPPERCASE. PATH, for example, stores the list of directories the shell searches through to find executables.

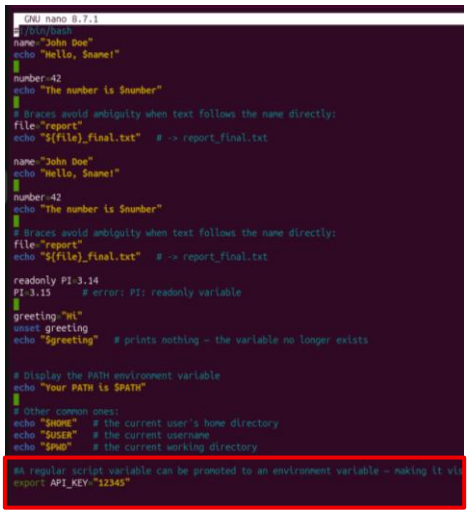
```
# Display the PATH environment variable
echo "Your PATH is $PATH"

# Other common ones:
echo "$HOME"    # the current user's home directory
echo "$USER"    # the current username
echo "$PWD"     # the current working directory
```



A regular script variable can be promoted to an environment variable — making it visible to any program the script launches — with export.

```
export API_KEY="12345"
```



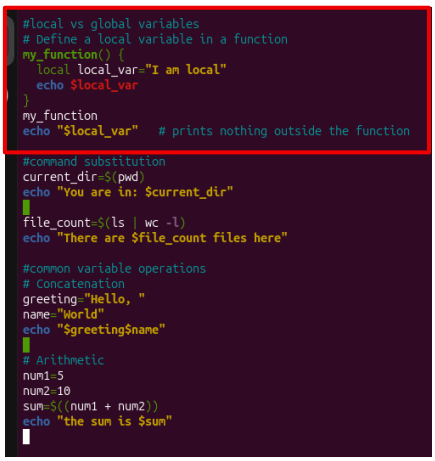
3.4 Special / Built-in Variables

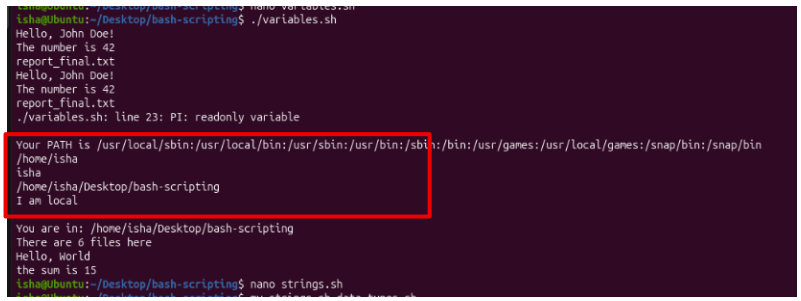
Variable	Meaning
$?	Exit status of the last command run
$$	Process ID (PID) of the current shell/script
$0	Name of the current script
$1 … $9	Positional arguments passed to the script
$#	Number of arguments passed

3.5 Local vs. Global Variables

A variable declared with the local keyword inside a function exists only within that function and disappears once the function finishes. Without local, a variable is global and can be read or changed from anywhere in the script, including from other functions.

```
# Define a local variable in a function
my_function() {
  local local_var="I am local"
  echo $local_var
}
my_function
echo "$local_var"   # prints nothing outside the function
```



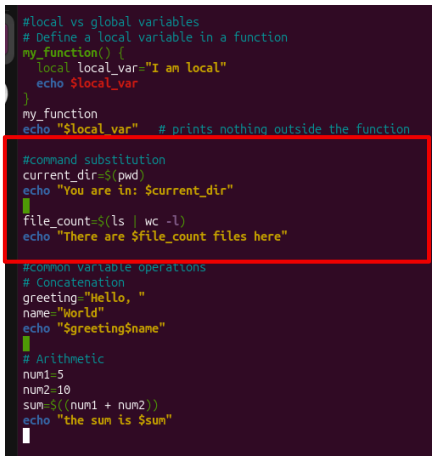


3.6 Command Substitution

Command substitution captures the output of a command and stores it in a variable, using $( ) (preferred) or backticks.

```
current_dir=$(pwd)
echo "You are in: $current_dir"
```

```
file_count=$(ls | wc -l)
echo "There are $file_count files here"
```



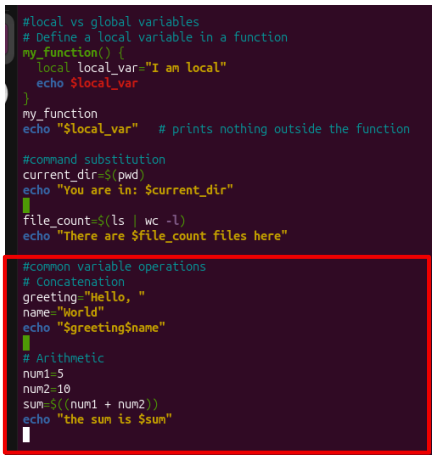


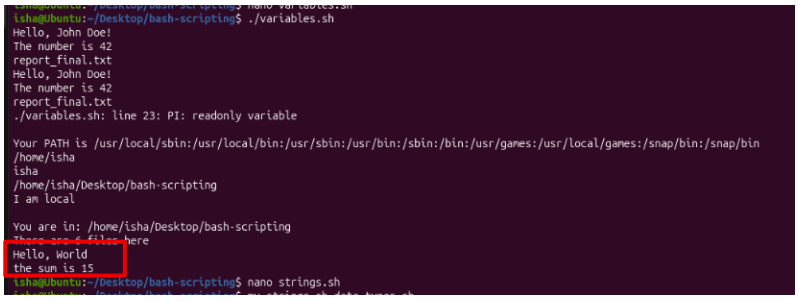
3.7 Common Variable Operations

Variables can be joined together (concatenation) or used in arithmetic expressions with $(( )) .

```
# Concatenation
greeting="Hello, "
name="World"
echo "$greeting$name"

# Arithmetic
num1=5
num2=10
sum=$((num1 + num2))
echo "The sum is $sum"
```





> **Common pitfall:**
> $name and "$name" usually look the same on screen, but if the value contains spaces or newlines, leaving out the quotes lets Bash split it into separate words. Get in the habit of quoting variables.

4. Bash Data Types

Bash does not enforce strict typing the way languages like Python or Java do, but values are still conceptually grouped into a few kinds of data: strings, numbers, indexed arrays, and associative arrays.

4.1 Strings

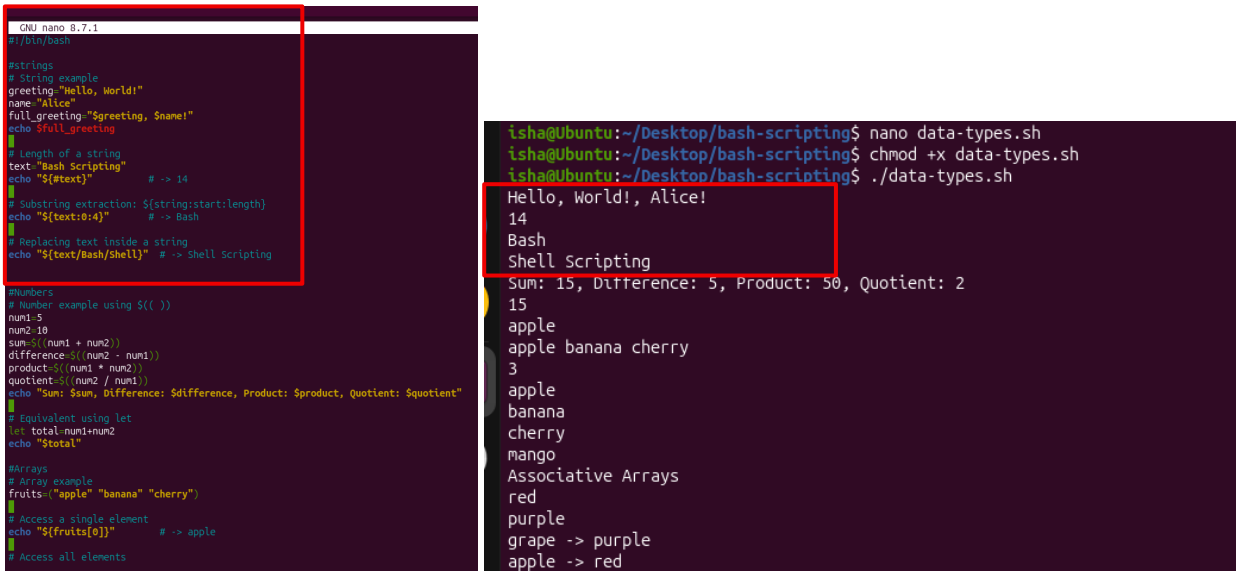
Strings are sequences of characters. They support operations such as concatenation, length lookup, and substring extraction.

```
# String example
greeting="Hello, World!"
name="Alice"
full_greeting="$greeting, $name!"
echo $full_greeting

# Length of a string
text="Bash Scripting"
echo "${#text}"          # -> 14

# Substring extraction: ${string:start:length}
echo "${text:0:4}"       # -> Bash

# Replacing text inside a string
echo "${text/Bash/Shell}"  # -> Shell Scripting
```



Single quotes vs. double quotes

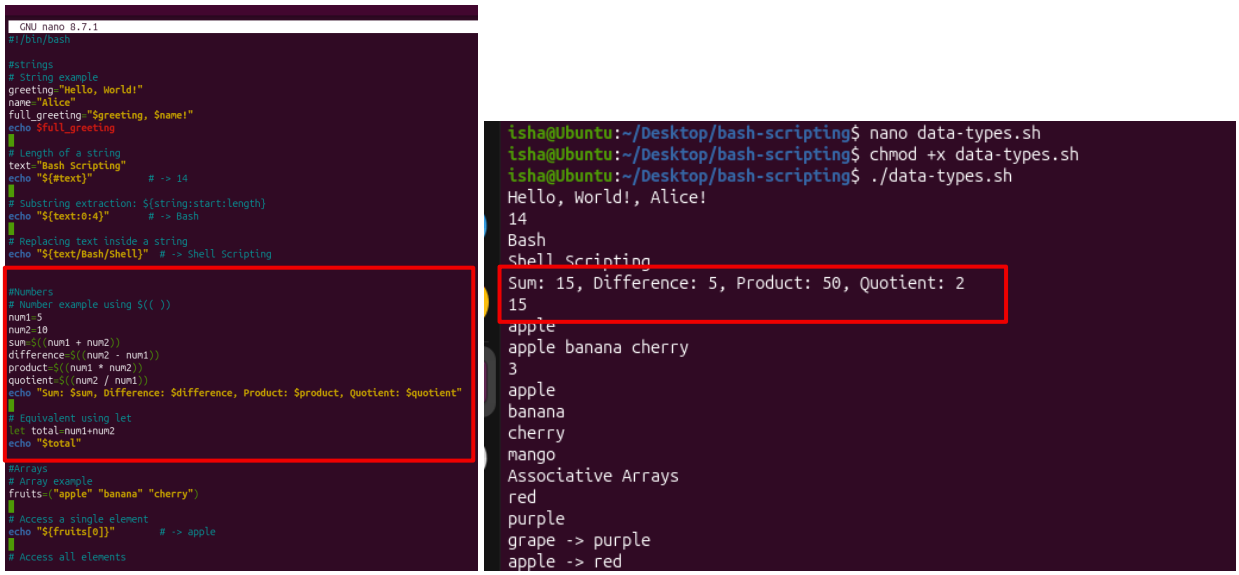
Quote type	Behaviour
'single quotes'	Everything inside is literal text — variables are NOT expanded.
"double quotes"	Variables and command substitutions ARE expanded (e.g. "$name").

4.2 Numbers

Bash natively supports integer arithmetic — addition, subtraction, multiplication, and division — using the $(( )) syntax, or the older let and expr commands.

```
# Number example using $(( ))
num1=5
num2=10
sum=$((num1 + num2))
difference=$((num2 - num1))
product=$((num1 * num2))
quotient=$((num2 / num1))
echo "Sum: $sum, Difference: $difference, Product: $product, Quotient: $quotient"

# Equivalent using let
let total=num1+num2
echo "$total"
```



> **Limitation:**
> Bash has no native support for floating-point (decimal) arithmetic — $((5/2)) gives 2, not 2.5. For decimal calculations, external tools such as bc or awk are used instead, e.g. echo "5/2" | bc -l.

4.3 Arrays (Indexed)

An array stores multiple values under a single variable name, with each element reachable by its numeric index (starting at 0). Arrays can be looped over, sliced, and extended easily.

```
# Array example
fruits=("apple" "banana" "cherry")
```

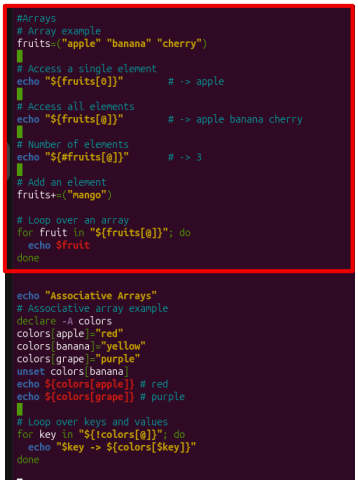
```
# Access a single element
echo "${fruits[0]}"        # -> apple

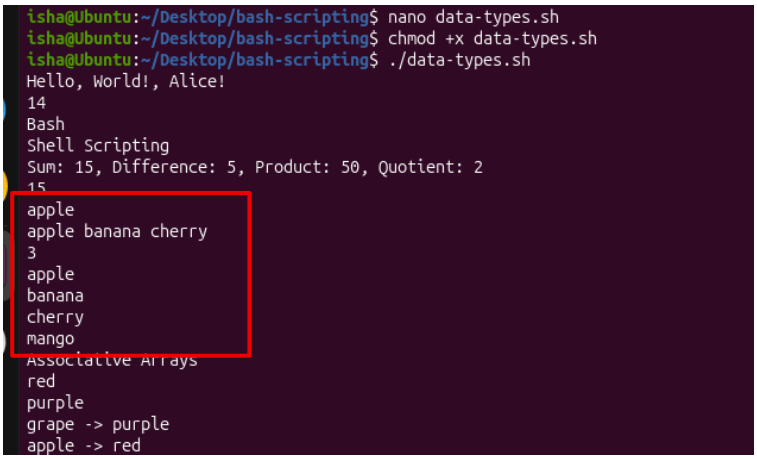
# Access all elements
echo "${fruits[@]}"        # -> apple banana cherry

# Number of elements
echo "${#fruits[@]}"       # -> 3

# Add an element
fruits+=("mango")

# Loop over an array
for fruit in "${fruits[@]}"; do
  echo $fruit
done
```



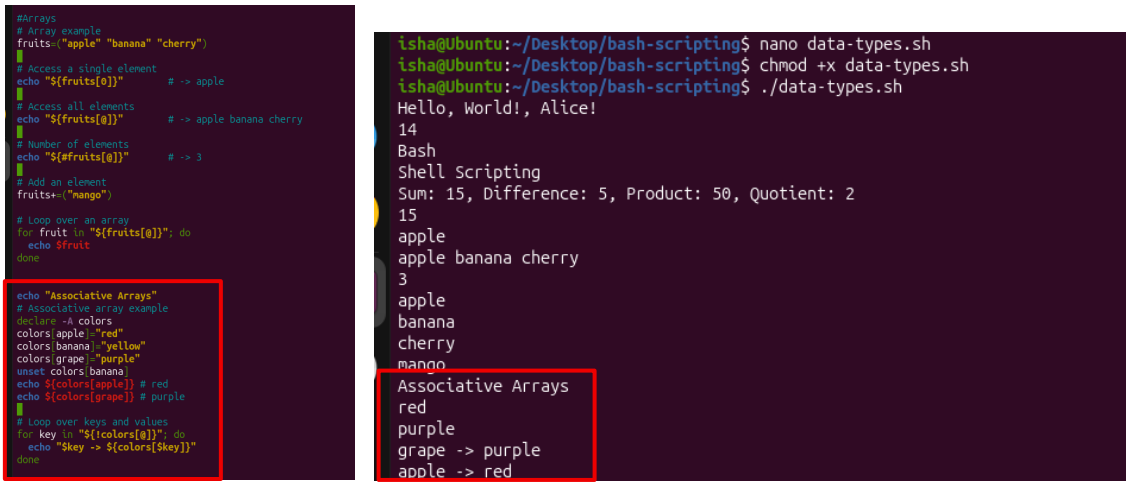


4.4 Associative Arrays

Associative arrays (declared with declare -A) map named keys to values, similar to a dictionary in other languages. Keys can be added or removed at any time, and are not kept in any guaranteed order.

```
# Associative array example
declare -A colors
colors[apple]="red"
colors[banana]="yellow"
colors[grape]="purple"
unset colors[banana]
echo ${colors[apple]} # red
echo ${colors[grape]} # purple

# Loop over keys and values
for key in "${!colors[@]}"; do
  echo "$key -> ${colors[$key]}"
done
```



```
#Arrays
# Array example
fruits=("apple" "banana" "cherry")

# Access a single element
echo "${fruits[0]}"        # -> apple

# Access all elements
echo "${fruits[@]}"        # -> apple banana cherry

# Number of elements
echo "${#fruits[@]}"       # -> 3

# Add an element
fruits+=("mango")

# Loop over an array
for fruit in "${fruits[@]}"; do
  echo $fruit
done

echo "Associative Arrays"
# Associative array example
declare -A colors
colors[apple]="red"
colors[banana]="yellow"
colors[grape]="purple"
unset colors[banana]
echo ${colors[apple]} # red
echo ${colors[grape]} # purple

# Loop over keys and values
for key in "${!colors[@]}"; do
  echo "$key -> ${colors[$key]}"
done
```

```
isha@Ubuntu:~/Desktop/bash-scripting$ nano data-types.sh
isha@Ubuntu:~/Desktop/bash-scripting$ chmod +x data-types.sh
isha@Ubuntu:~/Desktop/bash-scripting$ ./data-types.sh
Hello, World!, Alice!
14
Bash
Shell Scripting
Sum: 15, Difference: 5, Product: 50, Quotient: 2
15
apple
apple banana cherry
3
apple
banana
cherry
mango
Associative Arrays
red
purple
grape -> purple
apple -> red
```

4.5 Quick Type Reference

Data type	Declared as	Example
String	name="value"	city="Surat"
Number (integer)	name=value or declare -i	age=21
Indexed array	name=(v1 v2 v3)	days=(Mon Tue Wed)
Associative array	declare -A name	declare -A capitals

5. Bash Operators

Bash provides several categories of operators for comparing values, performing arithmetic, combining conditions, and testing files. Most comparisons appear inside test brackets: [ ] (the classic POSIX test command) or [[ ]] (a Bash-specific, more forgiving version).

5.1 Comparison Operators (numeric)

Operator	Meaning	Example
-eq	equal to	[$a -eq $b]
-ne	not equal to	[$a -ne $b]
-lt	less than	[$a -lt $b]
-le	less than or equal to	[$a -le $b]
-gt	greater than	[$a -gt $b]
-ge	greater than or equal to	[$a -ge $b]

5.2 String Comparison Operators

Operator	Meaning	Example
=	equal to	["$a" = "$b"]
!=	not equal to	["$a" != "$b"]
<	less than, ASCII order (needs [[ ]])	[["$a" < "$b"]]
>	greater than, ASCII order (needs [[ ]])	[["$a" > "$b"]]
-z	true if the string is empty	[-z "$a"]
-n	true if the string is NOT empty	[-n "$a"]

5.3 Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division (integer only)
%	Modulus (remainder)

Exponentiation is not built into basic arithmetic operators — Bash does support ** inside $(( )) for whole-number powers, but for anything beyond that, external tools such as bc or awk are used.

```
echo $((2 ** 5))     # -> 32
```

5.4 Logical Operators

Operator	Meaning	Example
&&	Logical AND — runs the next command only if the first succeeded	mkdir data && cd data
||	Logical OR — runs the next command only if the first failed	cd data || echo "failed"
!	Logical NOT — inverts a condition	if [! -f file.txt]; then ...

5.5 File Test Operators

Operator	Checks whether...
-e	a file or directory exists

Operator	Checks whether...
-d	a directory exists
-f	it is a regular file
-s	a file exists and is not empty
-r	a file is readable
-w	a file is writable
-x	a file is executable

```
if [ -f "notes.txt" ]; then
  echo "notes.txt exists"
fi
```

5.6 [ ] vs [[ ]]

[ ] is the traditional POSIX test command and works in every shell, but it requires careful quoting and does not support pattern matching or && / || directly inside it. [[ ]] is a Bash extension that is more forgiving about quoting and supports extra features such as pattern matching and combined conditions with && and ||.

```
# [[ ]] allows && directly inside the brackets
if [[ $num -gt 0 && $num -lt 100 ]]; then
  echo "In range"
fi
```

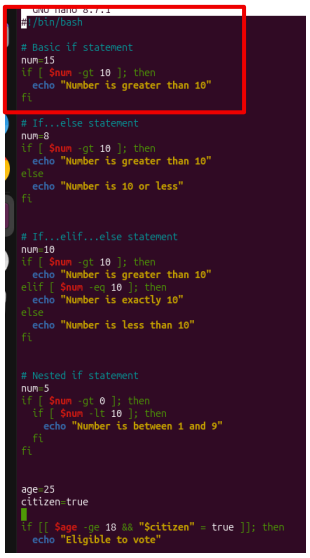
6. If...Else Statements

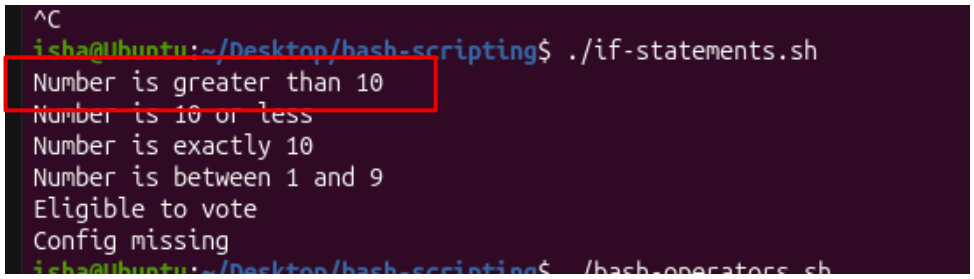
Conditional statements let a script branch its behaviour based on whether a condition is true or false. The condition itself is usually built from the comparison and file-test operators covered in Section 5.

6.1 If Statement

The condition is written inside square brackets [ ], and the block is closed with fi ("if" spelled backward). Note the semicolon before then, and the required spaces around the brackets.

```
# Basic if statement
num=15
if [ $num -gt 10 ]; then
  echo "Number is greater than 10"
fi
```

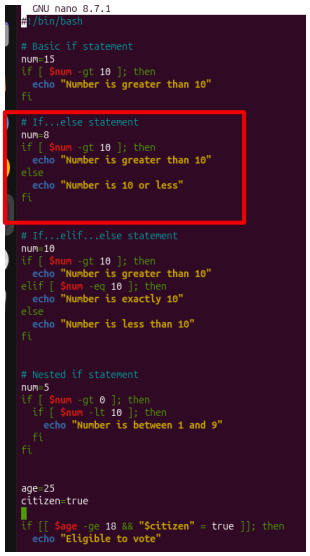


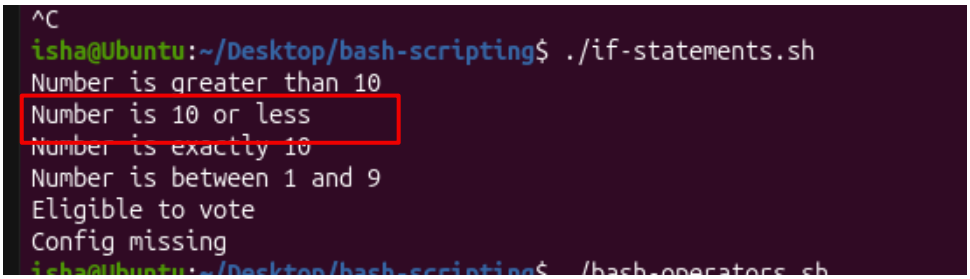


6.2 If...Else Statement

An else block runs when the condition is false, giving the script two possible paths.

```
# If...else statement
num=8
if [ $num -gt 10 ]; then
  echo "Number is greater than 10"
else
  echo "Number is 10 or less"
fi
```

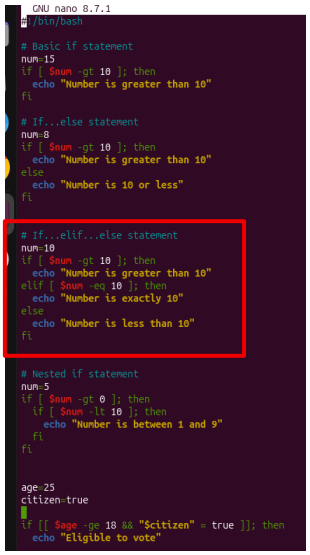


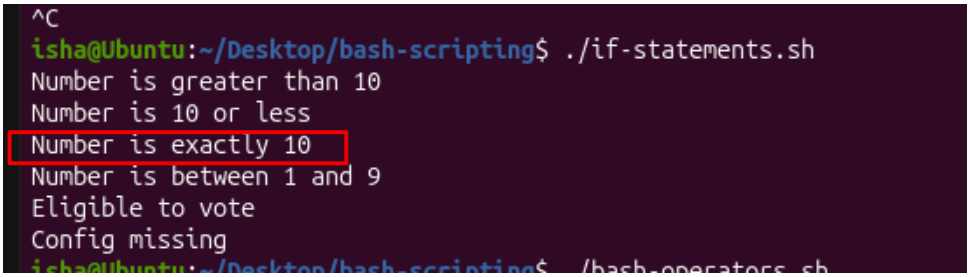


6.3 Elif Statement

elif ("else if") checks additional conditions in sequence when the previous ones are false. Any number of elif blocks can be chained before a final else.

```
# If...elif...else statement
num=10
if [ $num -gt 10 ]; then
  echo "Number is greater than 10"
elif [ $num -eq 10 ]; then
  echo "Number is exactly 10"
else
  echo "Number is less than 10"
fi
```

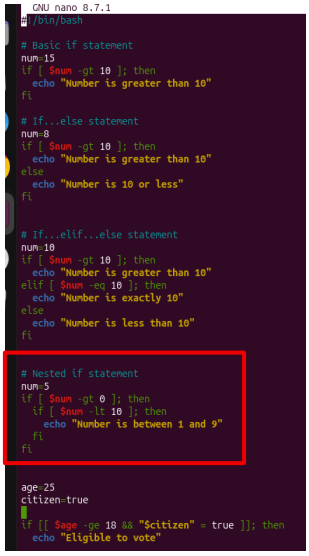


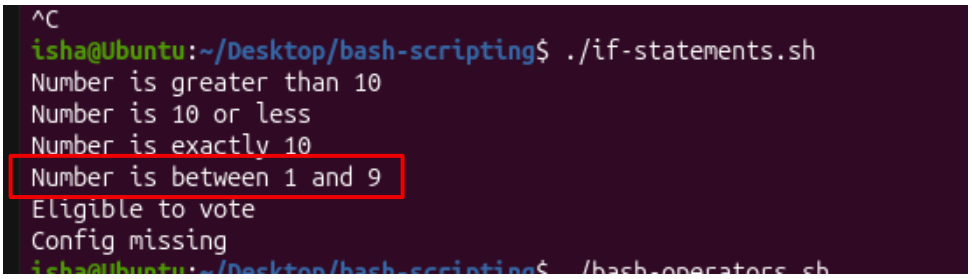


6.4 Nested If Statements

An if block can be placed inside another if block for more complex logic. Every if must be closed with its own fi, and indentation (though not required by Bash) is strongly recommended for readability.

```
# Nested if statement
num=5
if [ $num -gt 0 ]; then
  if [ $num -lt 10 ]; then
    echo "Number is between 1 and 9"
  fi
fi
```





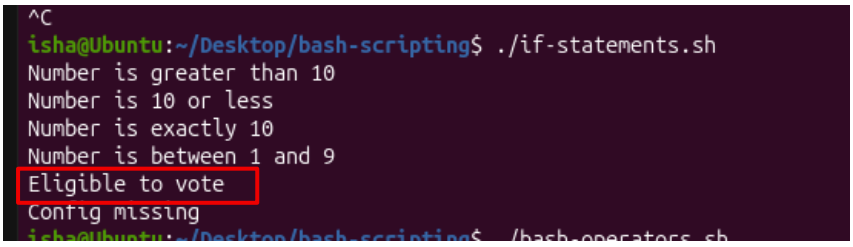
6.5 Combining Conditions

Multiple conditions can be combined with -a (AND) / -o (OR) inside [ ], or more readably with && / || inside [[ ]].

```
age=25
citizen=true

if [[ $age -ge 18 && "$citizen" = true ]]; then
  echo "Eligible to vote"
fi
```



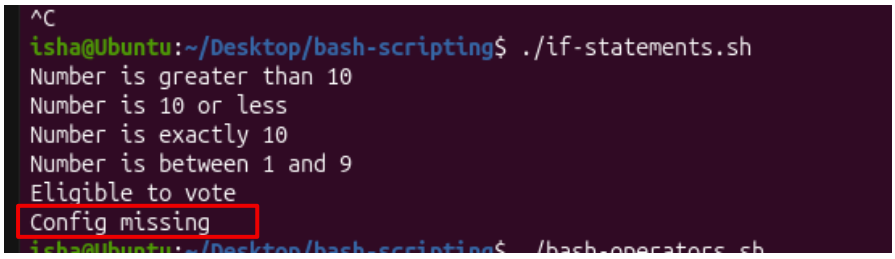


6.6 Short-hand Conditionals

A simple if/else with a single action on each side can sometimes be written more compactly using && and ||, which run the right-hand command only if the left-hand command succeeded / failed respectively.

```
[ -f "config.txt" ] && echo "Config found" || echo "Config missing"
```





> **Common pitfall:**
> Forgetting the spaces inside [$num -gt 10] (e.g. writing [$num-gt10]) is one of the most frequent syntax errors for beginners — the brackets are actually a command and need spaces like any other command.

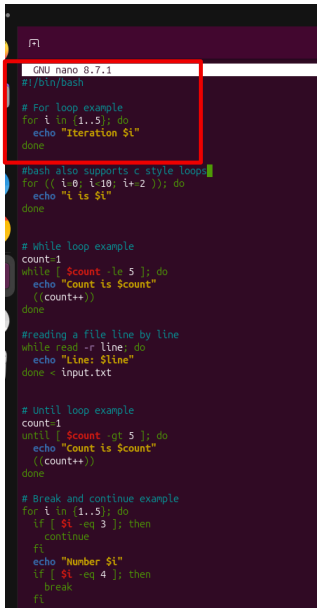
7. Bash Loops

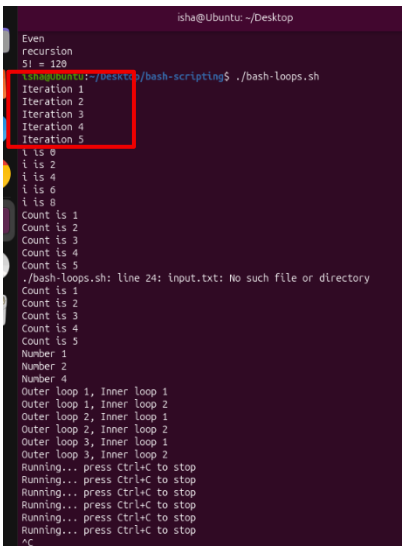
Loops repeat a block of commands, either a fixed number of times, over a list of items, or until/while a condition holds.

7.1 For Loop

A for loop iterates over a list of items or a range of numbers, running the block once per item.

```
# For loop example
for i in {1..5}; do
  echo "Iteration $i"
done
```

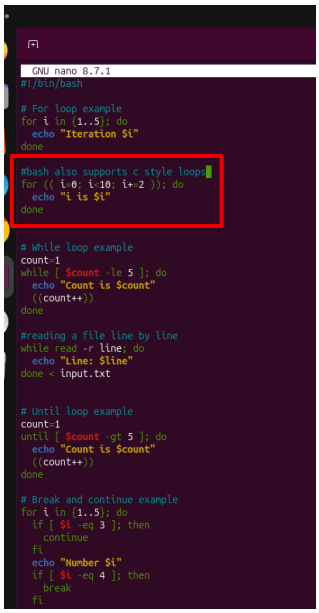


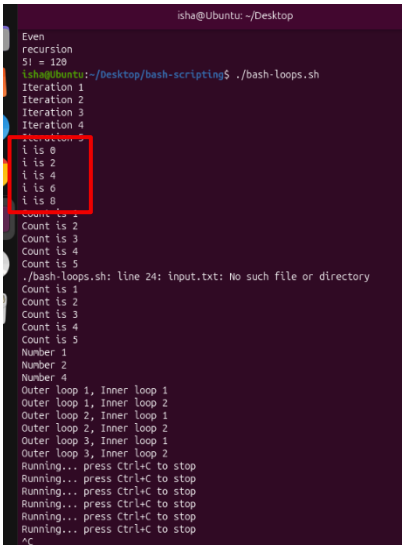


C-style for loop

Bash also supports a C-style for loop, useful when a custom step size is needed.

```
for (( i=0; i<10; i+=2 )); do
  echo "i is $i"
done
```

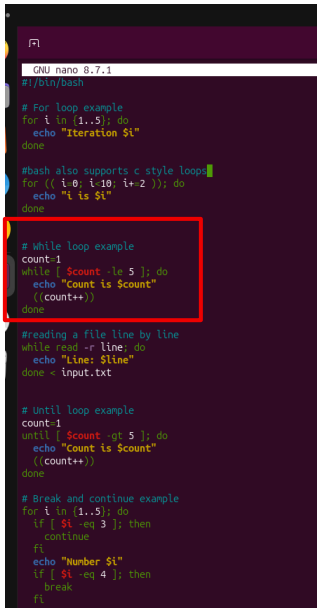


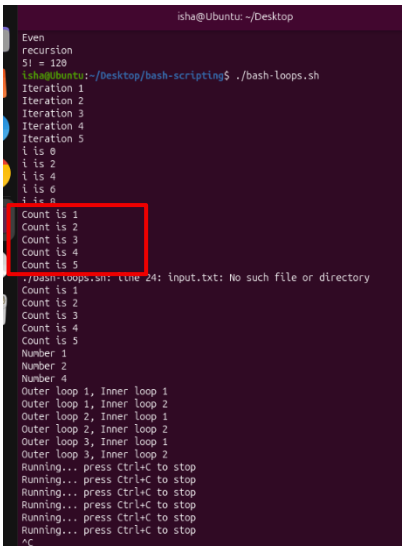


7.2 While Loop

A while loop keeps running as long as its condition stays true.

```
# While loop example
count=1
while [ $count -le 5 ]; do
  echo "Count is $count"
  ((count++))
done
```

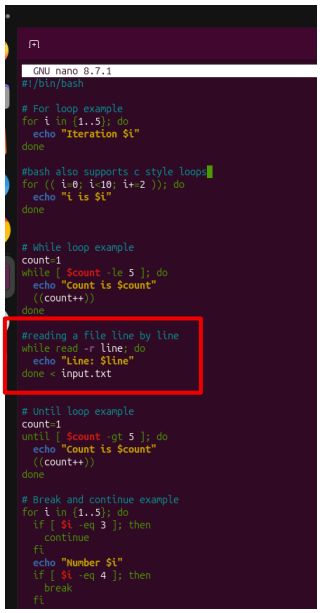




Reading a file line by line

A very common real-world use of while is reading a file one line at a time with the read command.

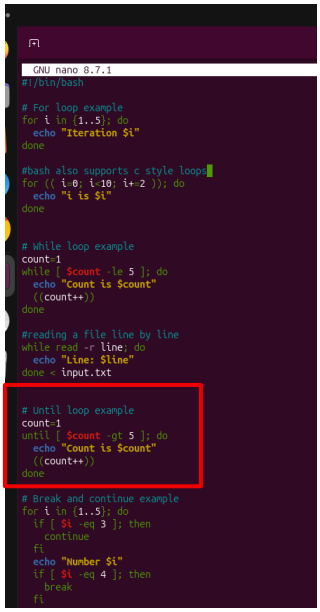
```
while read -r line; do
  echo "Line: $line"
done < input.txt
```

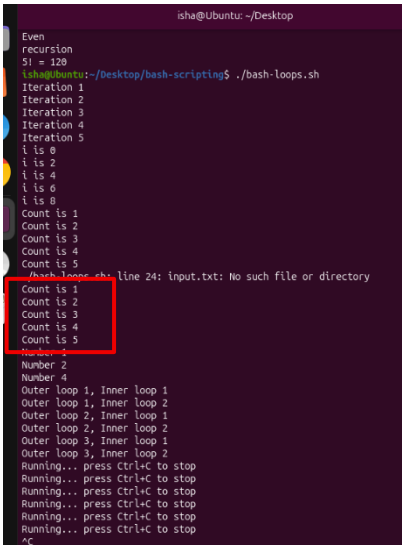


7.3 Until Loop

An until loop is the inverse of a while loop — it keeps running until its condition becomes true.

```
# Until loop example
count=1
until [ $count -gt 5 ]; do
  echo "Count is $count"
  ((count++))
done
```

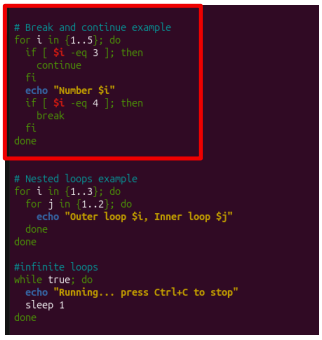


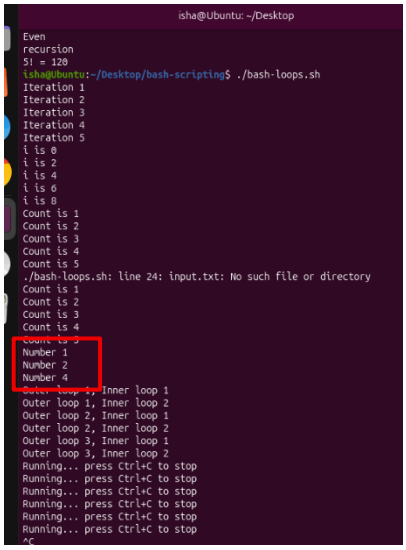


7.4 Break and Continue

break exits a loop immediately, while continue skips the rest of the current iteration and moves to the next one. Both can optionally take a number (break 2, continue 2) to affect an outer loop when loops are nested.

```
# Break and continue example
for i in {1..5}; do
  if [ $i -eq 3 ]; then
    continue
  fi
  echo "Number $i"
  if [ $i -eq 4 ]; then
    break
  fi
done
```

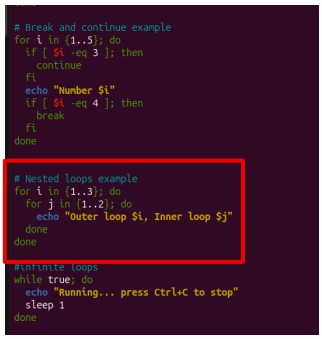


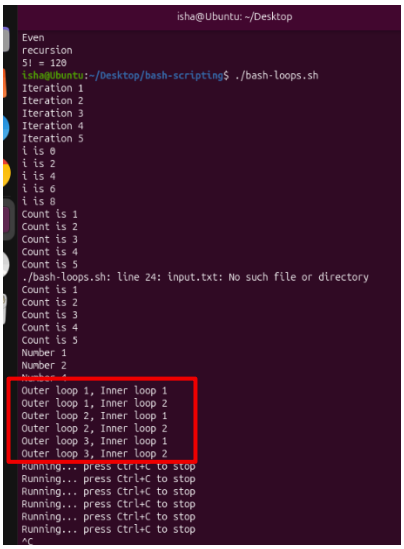


7.5 Nested Loops

A loop can be placed inside another loop for more complex iteration patterns, such as working through a grid or a table. Each loop needs its own done.

```
# Nested loops example
for i in {1..3}; do
  for j in {1..2}; do
    echo "Outer loop $i, Inner loop $j"
  done
done
```

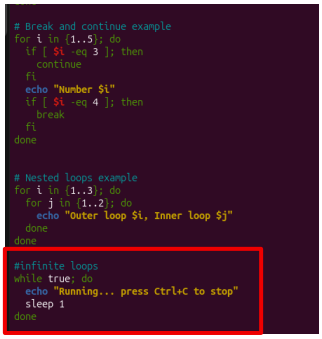


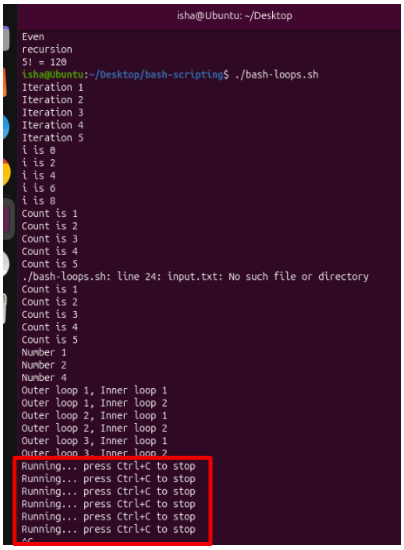


7.6 Infinite Loops

A loop with a condition that is always true runs forever unless stopped from inside (usually with break) or interrupted manually (Ctrl+C). These are common for scripts that poll or monitor something continuously.

```
while true; do
  echo "Running... press Ctrl+C to stop"
  sleep 1
done
```





7.7 Loop Types at a Glance

Loop	Runs while...
for	iterating over a fixed list or range
while	a condition remains TRUE
until	a condition remains FALSE (i.e. stops once it becomes true)

8. Bash Functions

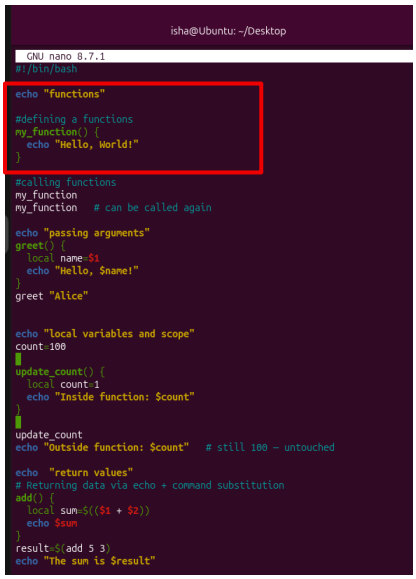
Functions group commands under a name so that a block of logic can be reused instead of repeated, which keeps scripts shorter and easier to maintain.

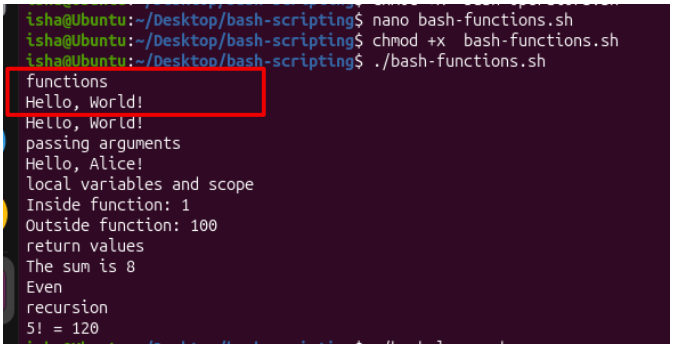
8.1 Defining a Function

A function name is followed by parentheses, with the body enclosed in curly braces. Descriptive function names are good practice, and the function must be defined before it is called in the script.

```
my_function() {
```

```
  echo "Hello, World!"
}
```

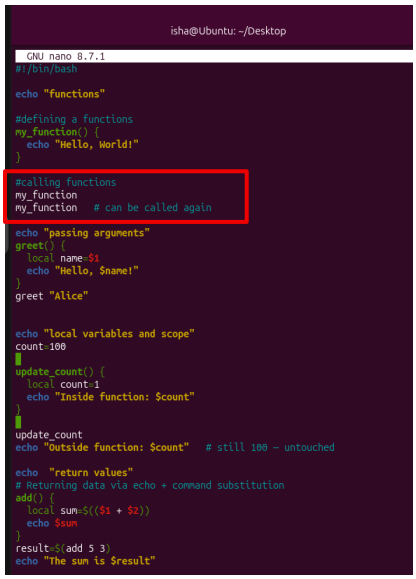


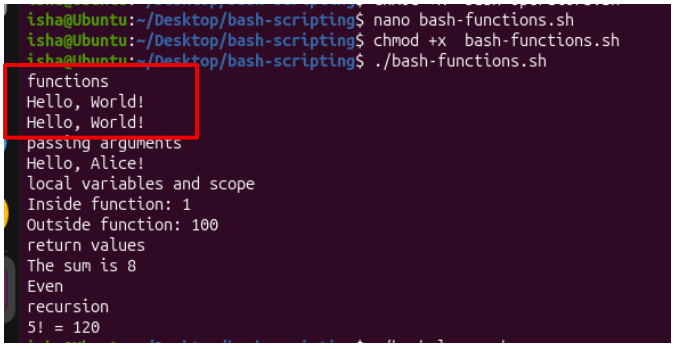


8.2 Calling a Function

A function is executed simply by writing its name; it can be called as many times as needed, from anywhere after its definition.

```
my_function
my_function   # can be called again
```

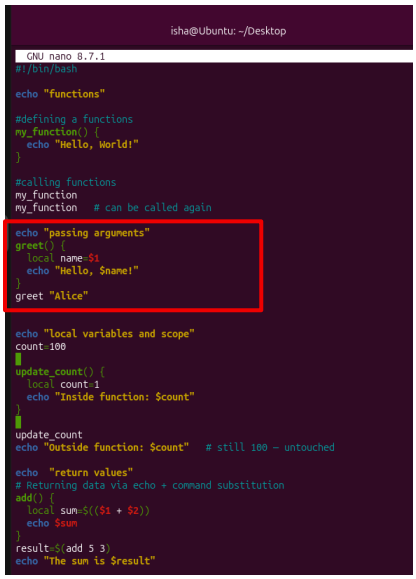


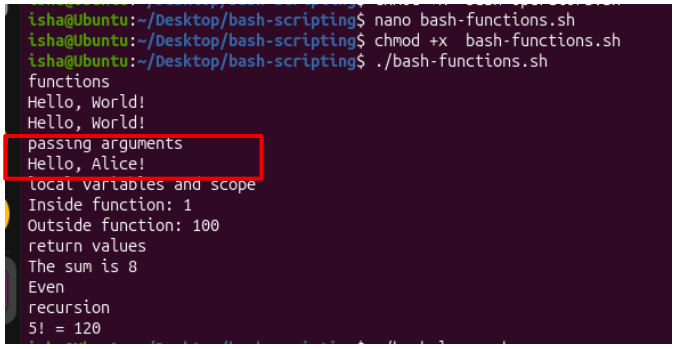


8.3 Passing Arguments

Functions accept arguments the same way scripts do — through positional parameters ($1, $2, ...) local to that function call, with $# giving the count of arguments received.

```
greet() {
  local name=$1
  echo "Hello, $name!"
}
greet "Alice"
```





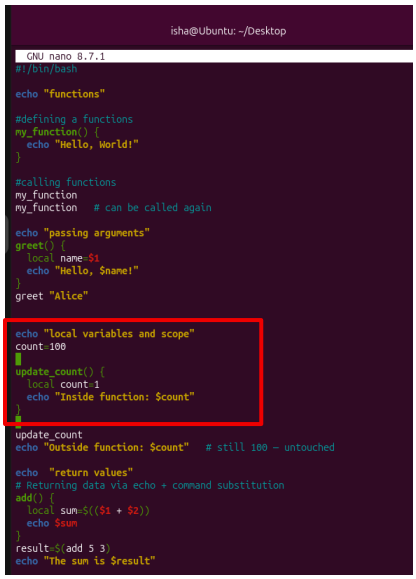
8.4 Local Variables and Scope

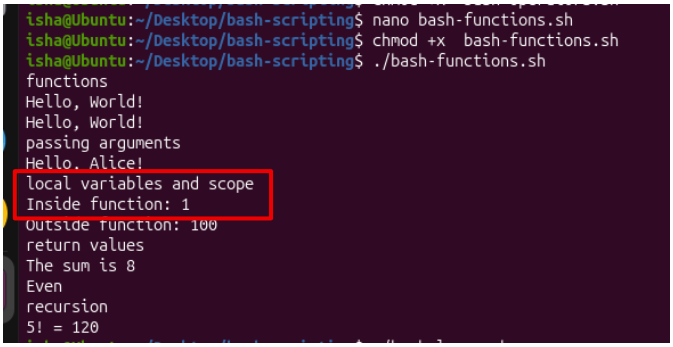
Using local inside a function keeps a variable confined to that function, preventing it from accidentally overwriting a global variable of the same name elsewhere in the script.

```
count=100

update_count() {
  local count=1
  echo "Inside function: $count"
}

update_count
echo "Outside function: $count"    # still 100 — untouched
```

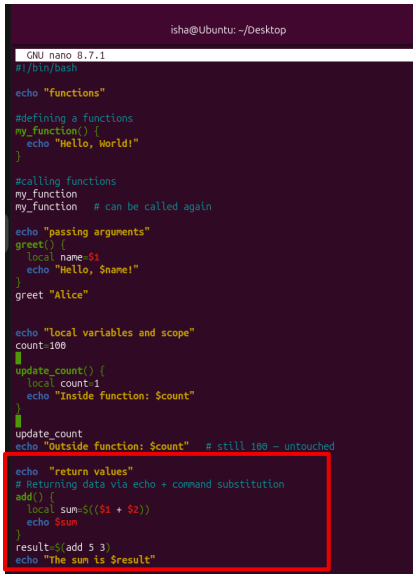


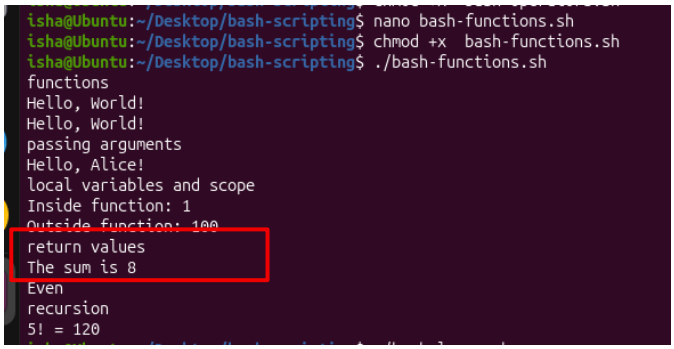


8.5 Return Values

Bash functions don't return values the way functions do in other languages. There are two common approaches: echo the result and capture it with command substitution, or use return to send back a small numeric status code (0–255, typically used for success/failure).

```
# Returning data via echo + command substitution
add() {
  local sum=$(($1 + $2))
  echo $sum
}
result=$(add 5 3)
echo "The sum is $result"
```

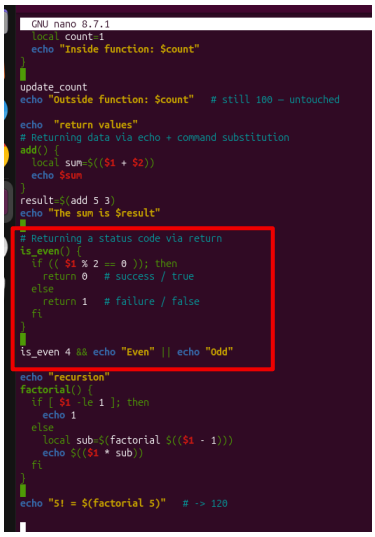


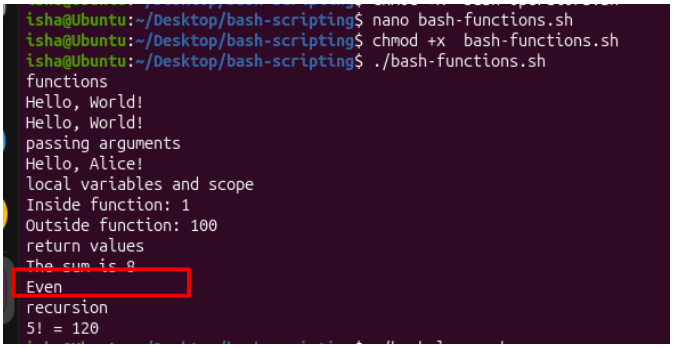



```
# Returning a status code via return
is_even() {
```

```
  if (( $1 % 2 == 0 )); then
    return 0   # success / true
  else
    return 1   # failure / false
  fi
}

is_even 4 && echo "Even" || echo "Odd"
```





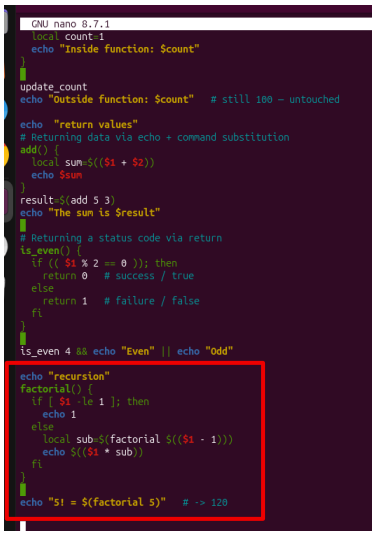
> **Note:**
> return only accepts whole numbers from 0-255 and is meant for status codes, not data. Use echo + $( ) whenever the function needs to hand back text or a computed value.

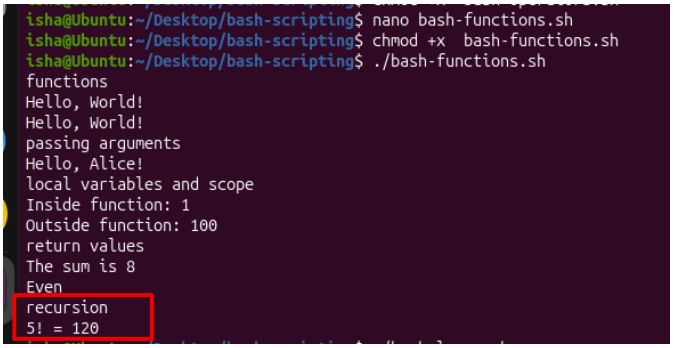
8.6 Recursion

A function can call itself, which is useful for problems that break down into smaller versions of themselves — as long as there is a base case to stop the recursion.

```
factorial() {
  if [ $1 -le 1 ]; then
    echo 1
  else
    local sub=$(factorial $(($1 - 1)))
    echo $(($1 * sub))
  fi
}

echo "5! = $(factorial 5)"   # -> 120
```





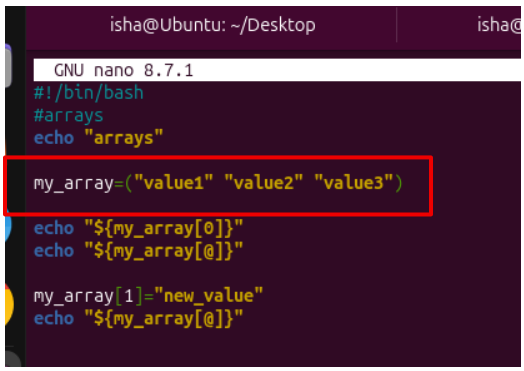
9. Bash Arrays

An array stores multiple values in a single variable, which makes managing related pieces of data far easier than juggling a separate variable for each one.

9.1 Creating Arrays

An array is created by listing its values inside parentheses, separated by spaces. Each value is indexed automatically, starting from zero. Using a descriptive name for the array helps make its purpose clear.

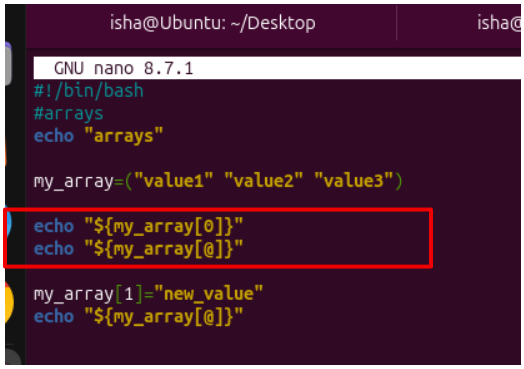
```
# Create an array
my_array=("value1" "value2" "value3")
```

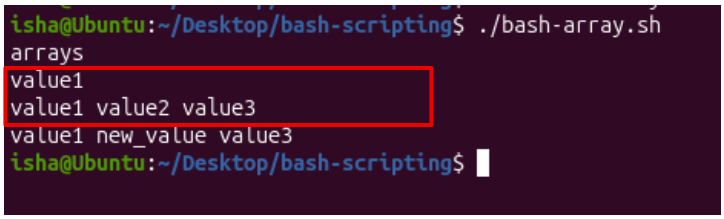


9.2 Accessing Array Elements

An element is read by placing its index inside square brackets after the array name.

```
echo ${my_array[0]}
```



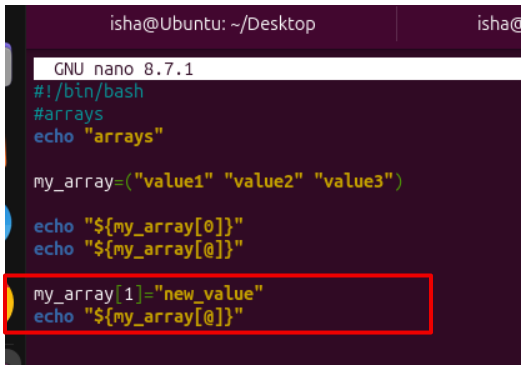


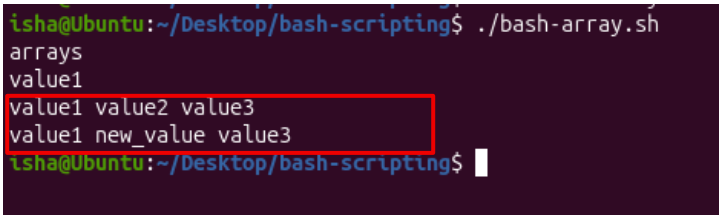
> **Note:**
> Bash arrays start at index 0, so ${my_array[0]} is the first element. Other shells differ: in zsh, the default shell on macOS, arrays start at index 1.

9.3 Modifying Array Elements

An existing element can be changed by assigning a new value at its index.

```
my_array[1]="new_value"
```





10. Bash crontab — Scheduling Tasks

cron is a time-based job scheduler built into Unix-like operating systems. It runs background jobs, called cron jobs, automatically at specified intervals. There is no direct “cron” command to run by hand — instead, crontab is the command used to create and manage the schedule.

10.1 The crontab Command

crontab manages a per-user text file listing commands to run on a schedule. The basic syntax is:

```
crontab [options]
```

Option	Meaning
-e	Edit the crontab file for the current user
-l	List the crontab entries for the current user
-r	Remove the crontab file for the current user

10.2 Cron Job Format

Each line in a crontab file represents one scheduled task and follows a fixed five-field pattern, followed by the command to run:

```
* * * * * command_to_execute
```

Field	Allowed range
Minute	0-59
Hour	0-23
Day of Month	1-31
Month	1-12
Day of Week	0-7 (0 and 7 are both Sunday)

Each asterisk can be replaced with a specific value or a range to restrict the job to particular minutes, hours, days, months, or weekdays.

10.3 Example: Scheduling a Task

To run a script every day at midnight, the minute and hour fields are set to 0 and the rest are left as asterisks:

```
0 0 * * * /path/to/script.sh
```

This entry runs /path/to/script.sh every day at 00:00 (midnight).

10.4 Common Uses

- Automating system maintenance tasks, like backups and updates.
- Scheduling scripts to run at specific times or intervals.
- Performing regular monitoring and reporting tasks.

11. More about Cron

cron is a time-based job scheduler built into Unix-like operating systems such as Linux. It runs commands or scripts automatically, either at a specific time or at regular intervals, without needing anyone to trigger them by hand. A task scheduled this way is called a cron job.

11.1 Why Use Cron?

Cron is useful anywhere a task needs to repeat on a schedule instead of being run manually every time. Typical examples include:

- Running backup scripts automatically
- Checking disk space periodically
- Monitoring system resources
- Running database backups
- Cleaning up temporary files
- Generating reports
- Running routine maintenance scripts
- Checking whether a service is still running
- Triggering deployment or automation scripts

For example, instead of manually running a backup script every day:

```
./backup.sh
```

cron can be told to run that same script automatically every night at 11:00 PM — without anyone remembering to do it.

How it works, step by step

```
Cron scheduler
     ↓
Checks the schedule
     ↓
Current time matches an entry
     ↓
Runs the command / script
     ↓
Task completes automatically
```

11.2 The Cron Service

cron itself runs as a background service (a daemon). On Ubuntu this service is simply called cron. Its status can be checked at any time:

```
isha@Ubuntu:~$ systemctl status cron
Active: active (running)
```

That line confirms the service is running and ready to execute scheduled jobs.

Action	Command
Start cron	sudo systemctl start cron
Stop cron	sudo systemctl stop cron
Restart cron	sudo systemctl restart cron
Reload systemd configuration	sudo systemctl daemon-reload
Check whether cron starts automatically at boot	systemctl is-enabled cron

11.3 The crontab Command

crontab (short for “cron table”) is the command used to create, view, and manage a user's cron jobs. A crontab is simply a list of scheduled commands. Basic syntax:

```
crontab [options]
```

Option	Description
-e	Edit the current user's crontab
-l	List the current user's cron jobs
-r	Remove the current user's entire crontab
-u	Specify another user (when permitted)

11.3.1 Listing Cron Jobs

```
isha@Ubuntu:~/cron-practice$ crontab -l
* * * * * /home/isha/cron-practice/test.sh
```

If no jobs have been created yet, the output instead looks like this — which simply means the user has no personal crontab so far:

```
isha@Ubuntu:~/cron-practice$ crontab -l
no crontab for isha
```

11.3.2 Editing Cron Jobs

crontab -e opens the current user's crontab in a text editor so jobs can be added, changed, or removed:

```
isha@Ubuntu:~/cron-practice$ crontab -e
# add this line, save, and exit:
* * * * * /home/isha/cron-practice/test.sh
```

Saving the file and exiting the editor registers the job with cron.

11.3.3 Removing the Crontab

```
isha@Ubuntu:~/cron-practice$ crontab -r
```

> **Caution:**
> crontab -r deletes the entire crontab for that user — every job is removed at once, not just one. During normal practice it is safer to run crontab -e and delete only the specific line that is no longer needed.

11.4 Cron Job Format

Every cron job follows the same fixed format — five schedule fields followed by the command to run:

```
MINUTE   HOUR   DAY_OF_MONTH   MONTH   DAY_OF_WEEK   COMMAND
```

For example:

```
0 9 * * 1-5 /home/isha/script.sh
```

The position of each field is what gives it meaning:

```
┌──────── Minute
│ ┌────── Hour
│ │ ┌──── Day of month
│ │ │ ┌── Month
│ │ │ │ ┌ Day of week
│ │ │ │ │
* * * * * command
```

11.5 The Five Cron Fields

Each field has its own allowed range of values:

Field	Allowed values	Notes
Minute	0-59	The minute of the hour the job runs at

Field	Allowed values	Notes
Hour	0-23	24-hour clock: 0 = 12 AM, 12 = 12 PM, 18 = 6 PM, 23 = 11 PM
Day of Month	1-31	A specific day within the month
Month	1-12	1 = January ... 12 = December
Day of Week	0-7	0 and 7 both mean Sunday; 1 = Monday ... 6 = Saturday

A few field examples:

```
30 * * * * command      # minute 30 of every hour
0 18 * * * command      # every day at 6 PM
0 9 15 * * command      # 9:00 AM on the 15th of the month
0 9 1 1 * command       # 9:00 AM on January 1st
0 9 * * 1 command       # every Monday at 9:00 AM
```

11.6 The Asterisk (*)

An asterisk in any field means “every possible value” for that field. So:

```
* * * * * command
```

means every minute, of every hour, of every day, of every month, on every day of the week — in other words, the command runs once every minute, all the time.

11.7 Practice: Running a Task Every Minute

Create a small script:

```
isha@Ubuntu:~/cron-practice$ nano test.sh
```

With the contents:

```
#!/bin/bash
echo "Cron ran at $(date)" >> ~/cron-practice/output.txt
```

Make it executable and test it manually before scheduling it:

```
isha@Ubuntu:~/cron-practice$ chmod +x test.sh
isha@Ubuntu:~/cron-practice$ ./test.sh
isha@Ubuntu:~/cron-practice$ cat output.txt
Cron ran at Wed Sep 16 12:12:40 PM IST 2026
```

Now register it with cron:

```
isha@Ubuntu:~/cron-practice$ crontab -e
# add this line, then save and exit:
* * * * * /home/isha/cron-practice/test.sh
```

After a few minutes, checking the log again shows a new entry appended every minute:

```
isha@Ubuntu:~/cron-practice$ cat output.txt
Cron ran at Wed Sep 16 12:12:40 PM IST 2026
Cron ran at Wed Sep 16 12:15:01 PM IST 2026
Cron ran at Wed Sep 16 12:16:01 PM IST 2026
Cron ran at Wed Sep 16 12:24:03 PM IST 2026
Cron ran at Wed Sep 16 12:24:52 PM IST 2026
Cron ran at Wed Sep 16 12:25:33 PM IST 2026
```

> **Tip:**
> If a job's output file shows "No such file or directory", double-check the path used inside the script — cron runs with a much smaller environment than an interactive terminal, so relative paths that work when typed by hand can fail inside a scheduled job. Using the full path (e.g. /home/isha/cron-practice/output.txt) avoids this.

11.8 Scheduling Tasks at Specific Times

Fixing the minute and hour fields, and leaving the rest as *, schedules a job for the same time every day:

Schedule	Cron expression
Every day at midnight	0 0 * * * command
Every day at 9 AM	0 9 * * * command
Every day at 6:30 PM	30 18 * * * command
Every day at 11:45 PM	45 23 * * * command

Reading 0 0 * * * command field by field: minute 0, hour 0, every day of the month, every month, every weekday — so it runs once, exactly at 12:00 AM, every day.

11.9 Running a Job Every N Minutes

The slash (/) operator defines a step, or interval, within a field — useful for “every N minutes” style schedules.

Schedule	Cron expression
Every 5 minutes	*/5 * * * * command

Schedule	Cron expression
Every 10 minutes	*/10 * * * * command
Every 15 minutes	*/15 * * * * command
Every 30 minutes	*/30 * * * * command

*/5 * * * * command runs at 10:00, 10:05, 10:10, 10:15, and so on — every fifth minute of every hour.

11.10 Running a Job Every Hour

Fixing the minute at 0 and leaving the hour as * runs a job once at the start of every hour:

```
0 * * * * command
```

This triggers at 09:00, 10:00, 11:00, 12:00, and so on, all day.

11.11 Running a Job Multiple Times (Comma)

A comma lists several specific values for a single field.

```
0 9,13,18 * * * command   # 9:00 AM, 1:00 PM, and 6:00 PM
0,30 9 * * * command      # 9:00 AM and 9:30 AM
```

11.12 Using Ranges (Hyphen)

A hyphen defines a continuous range of values for a field.

```
0 9-17 * * * command
```

This runs at the start of every hour from 9 AM through 5 PM: 9, 10, 11, 12, 1, 2, 3, 4, and 5 o'clock.

11.13 Scheduling Jobs on Specific Days

Schedule	Cron expression
Every Monday at 9 AM	0 9 * * 1 command
Every Friday at 6 PM	0 18 * * 5 command
Every Saturday & Sunday at 10 AM	0 10 * * 0,6 command
Monday to Friday at 9 AM	0 9 * * 1-5 command

11.14 Scheduling on Specific Dates

Schedule	Cron expression
First day of every month, 9 AM	0 9 1 * * command
15th of every month, 10 AM	0 10 15 * * command
December 1st, 9 AM	0 9 1 12 * command

11.15 Common Cron Shortcuts

A few predefined shortcuts exist for the most common schedules, so the five fields don't need to be written out by hand:

Shortcut	Meaning
@reboot	Run once when cron starts / the system boots
@hourly	Run once every hour
@daily	Run once every day
@weekly	Run once every week
@monthly	Run once every month
@yearly	Run once every year

For example, instead of writing:

```
0 0 * * * /home/isha/backup.sh
```

the shortcut form can be used instead:

```
@daily /home/isha/backup.sh
```

11.16 @reboot

@reboot runs a command once, automatically, whenever the system starts and cron processes the user's crontab — useful for starting user-level scripts or applications after a reboot.

```
@reboot /home/isha/startup.sh
```

11.17 Combining Cron with a Bash Script

Cron becomes especially powerful when paired with a script that does several things at once. For example, a small system-information logger:

```
isha@Ubuntu:~/cron-practice$ nano system-info.sh
```

With the contents:

```
#!/bin/bash
echo "====================" >> ~/cron-practice/system.log
echo "Time: $(date)" >> ~/cron-practice/system.log
echo "User: $(whoami)" >> ~/cron-practice/system.log
echo "Hostname: $(hostname)" >> ~/cron-practice/system.log

echo "Disk Usage:" >> ~/cron-practice/system.log
df -h >> ~/cron-practice/system.log

echo "Memory Usage:" >> ~/cron-practice/system.log
free -h >> ~/cron-practice/system.log
```

Make it executable, test it once manually, then check the log it produced:

```
isha@Ubuntu:~/cron-practice$ chmod +x system-info.sh
isha@Ubuntu:~/cron-practice$ ./system-info.sh
isha@Ubuntu:~/cron-practice$ cat system.log
====================
Time: Wed Sep 16 10:10:02 IST 2026
User: isha
Hostname: isha-ubuntu
Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   18G   30G  38% /
Memory Usage:
               total        used        free
Mem:           7.7Gi       2.1Gi       3.9Gi
```

Finally, schedule it to run automatically every 10 minutes:

```
*/10 * * * * /home/isha/cron-practice/system-info.sh
```

This combines a Bash script, ordinary Linux commands, and cron into one simple, automated system-monitoring routine.

11.18 Cron for Automated Backups

A very common real-world use of cron is running backups on a schedule. First, some practice data:

```
isha@Ubuntu:~/cron-practice$ mkdir -p data
isha@Ubuntu:~/cron-practice$ echo "Important data" > data/file.txt
```

Then a backup script that copies that folder into a timestamped backup directory:

```
isha@Ubuntu:~/cron-practice$ nano backup.sh
```

```
#!/bin/bash
mkdir -p ~/cron-practice/backups
cp -r ~/cron-practice/data \
~/cron-practice/backups/data-$(date +%Y%m%d-%H%M%S)
```

```
isha@Ubuntu:~/cron-practice$ chmod +x backup.sh
isha@Ubuntu:~/cron-practice$ ./backup.sh
isha@Ubuntu:~/cron-practice$ ls backups
data-20260916-122500
```

Scheduled to run automatically every 30 minutes:

```
*/30 * * * * /home/isha/cron-practice/backup.sh
```

11.19 Cron for Log Monitoring

Cron can also periodically scan a log file for problems. A sample log entry to search for:

```
isha@Ubuntu:~/cron-practice$ echo "ERROR: Test error" >> application.log
```

And a small script that checks it and records what it finds:

```
isha@Ubuntu:~/cron-practice$ nano log-check.sh
```

```
#!/bin/bash
echo "===== LOG CHECK $(date) =====" >> ~/cron-practice/log-check.log
grep -i "error" ~/cron-practice/application.log \
>> ~/cron-practice/log-check.log
```

```
isha@Ubuntu:~/cron-practice$ chmod +x log-check.sh
isha@Ubuntu:~/cron-practice$ ./log-check.sh
isha@Ubuntu:~/cron-practice$ cat log-check.log
===== LOG CHECK Wed Sep 16 10:20:01 IST 2026 =====
ERROR: Test error
```

Scheduled every 10 minutes, this becomes a simple form of automated log monitoring:

```
*/10 * * * * /home/isha/cron-practice/log-check.sh
```

11.20 Cron Output and Redirection

A cron job can produce both normal output and error output, and both can be redirected to a file so nothing is lost:

```
* * * * * /home/isha/cron-practice/test.sh >> ~/cron-practice/output.log
```

>> appends the output to the file instead of overwriting it each time. To also capture error messages in the same file:

```
* * * * * /home/isha/cron-practice/test.sh >> ~/cron-practice/output.log 2>&1
```

Understanding 2>&1

Stream number	Meaning
0	Standard input
1	Standard output
2	Standard error

2>&1 means “send stream 2 (errors) to wherever stream 1 (normal output) is currently going.” It is a very common pattern in automation scripts and cron jobs so that both kinds of output land in one log file.

11.21 Checking Cron Logs

Cron's own activity can be inspected through the system journal:

```
isha@Ubuntu:~$ sudo journalctl -u cron
```

To see only recent activity:

```
isha@Ubuntu:~$ sudo journalctl -u cron --since "10 minutes ago"
```

To watch it continuously (press Ctrl+C to stop):

```
isha@Ubuntu:~$ sudo journalctl -u cron -f
```

On systems where cron writes to /var/log/syslog instead, the same information can be found with:

```
isha@Ubuntu:~$ sudo grep CRON /var/log/syslog
```

11.22 Cron Command Reference

Command	Purpose
crontab -l	List the current user's cron jobs
crontab -e	Edit the current user's cron jobs
crontab -r	Remove the current user's entire crontab
systemctl status cron	Check whether the cron service is running
sudo systemctl start cron	Start the cron service
sudo systemctl stop cron	Stop the cron service
sudo systemctl restart cron	Restart the cron service
sudo systemctl daemon-reload	Reload systemd configuration
sudo journalctl -u cron	View cron's service logs
sudo journalctl -u cron -f	Continuously monitor cron logs

11.23 Quick Reference: Common Cron Expressions

Requirement	Cron expression
Every minute	* * * * *
Every 5 minutes	*/5 * * * *
Every 10 minutes	*/10 * * * *
Every 15 minutes	*/15 * * * *
Every 30 minutes	*/30 * * * *
Every hour	0 * * * *
Every day at midnight	0 0 * * *
Every day at 9 AM	0 9 * * *
Every day at 6:30 PM	30 18 * * *
Every Monday at 9 AM	0 9 * * 1
Monday–Friday at 9 AM	0 9 * * 1-5
Saturday and Sunday at 10 AM	0 10 * * 0,6

Requirement	Cron expression
First day of every month, 9 AM	0 9 1 * *
15th of every month, 10 AM	0 10 15 * *
9 AM, 1 PM, and 6 PM	0 9,13,18 * * *

11.24 How to Read a Complex Cron Expression

Longer expressions are easiest to read one field at a time. Take:

```
*/5 9-17 * * 1-5 /home/isha/script.sh
```

Breaking it down field by field:

Field value	Meaning
*/5	Every 5 minutes
9-17	Between 9 AM and 5 PM
*	Every day of the month
*	Every month
1-5	Monday through Friday

Put together: the script runs every 5 minutes, between 9 AM and 5 PM, Monday through Friday.

11.25 Practice Exercises

Exercise 1 — Every Minute

Write a script that appends the text “Hello from cron” to ~/cron-practice/hello.txt, and schedule it to run every minute.

Exercise 2 — Every 5 Minutes

Write a script that records the current time using date, and schedule it with */5 * * * *.

Exercise 3 — System Information

Write a script that records the date, hostname, current user, disk usage, and memory usage (using date, hostname, whoami, df -h, and free -h). Schedule it every 15 minutes.

Exercise 4 — Backup

- Creates a backup directory.
- Copies a folder into that backup directory.
- Adds the current date and time to the backup's folder name.
- Runs automatically every 30 minutes.

Exercise 5 — Service Monitoring

Write a script that checks systemctl is-active cron, prints “Cron is running” if active and “Cron is NOT running” otherwise, and schedule it every 5 minutes.

Exercise 6 — Log Monitoring

Write a script that searches a log file for the word “error” using grep -i "error", saves the result to another file, and schedule it every 10 minutes.

11.26 Best Practices for Cron

1. Use absolute paths

Cron runs with a different, much smaller environment than an interactive terminal, so relying on the current directory can fail silently.

```
# Avoid:
* * * * * ./script.sh

# Prefer:
* * * * * /home/isha/cron-practice/script.sh
```

2. Make scripts executable

```
isha@Ubuntu:~/cron-practice$ chmod +x script.sh
isha@Ubuntu:~/cron-practice$ ./script.sh
```

3. Test manually before scheduling

Run the script directly first (./script.sh) and confirm it behaves correctly before adding it with crontab -e.

4. Log important output

```
*/10 * * * * /home/isha/script.sh >> /home/isha/script.log 2>&1
```

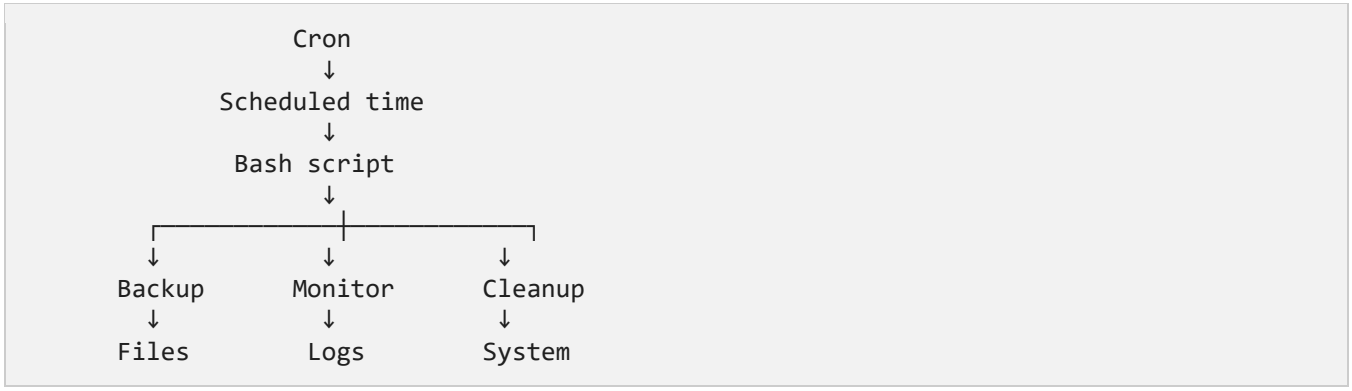
This makes troubleshooting a failed or misbehaving job much easier later.

5. Be careful with crontab -r

> **Reminder:**
> crontab -r removes a user's entire crontab in one go. Use crontab -e instead when only one job needs to be changed or removed.

11.27 Cron in DevOps

In a DevOps context, cron is a simple but effective way to automate recurring operational work — without needing a heavier scheduling system for everyday tasks.



Typical DevOps examples following this same pattern:

```
Daily backup → Bash script → Cron → Automatic execution

Every 5 minutes → Health-check script → Check service → Write result to log
```